

CS6650 HW7 — Part III Report

Serverless Order Processing: Lambda vs ECS Worker

Overview

Part III replaces the ECS SQS worker with an AWS Lambda function that subscribes directly to the same SNS topic (bypassing SQS). The goal is to observe cold start behavior and compare the cost and operational trade-offs between ECS and Lambda for order processing.

Item	Details
Function Name	hw7-order-processor
Runtime	provided.al2 (Go binary, custom runtime)
Trigger	SNS Topic: hw7-order-processing-events (direct subscription, bypasses SQS)
Timeout	30 seconds
Memory	128 MB configured, 20 MB actually used
Processing	Same 3-second simulated payment verification as ECS worker

Cold Start Observations

Six test orders were sent to /orders/async across multiple invocations. CloudWatch logs were examined for INIT_START and Init Duration fields, which are present only on cold start invocations.

Log streams (5) [Delete](#) [Create log](#)

By default, we only load the most recent log streams.

☐ Exact match ☐ Show expired [Info](#)

☐ Log stream	☐ Last event time
☐ 2026/03/15/[\$LATEST]c81af316263d4b698750f0e66828b79d	2026-03-15 13:58:26 (UTC-07:00)
☐ 2026/03/15/[\$LATEST]095e8054b614455ca744ebfea75d9c94	2026-03-15 13:58:26 (UTC-07:00)
☐ 2026/03/15/[\$LATEST]5260b89c5b234a59ad39828531ddbd00	2026-03-15 13:58:26 (UTC-07:00)
☐ 2026/03/15/[\$LATEST]0ded885c9d27489b8aa52c757552b4e5	2026-03-15 13:58:23 (UTC-07:00)
☐ 2026/03/15/[\$LATEST]b3f684ca9486436d8f8107afeb0ffe1f	2026-03-15 13:58:23 (UTC-07:00)

Figure L1 — CloudWatch Log Groups showing 5 separate log streams created simultaneously. Each SNS-triggered Lambda invocation spawned an independent container instance, all cold-starting at the same timestamp (13:58:23).

No older events at this moment. [Retry](#)

2026-03-15T13:58:23.017-07:00	INIT_START Runtime Version: provided:al2.v143 Runtime Version ARN: arn:aws:lambda:us-west-2::runtime:64c84f04bae0287749677ef420f09c0b2f4a3c7b1a638630bcc38cf04c7b	📄
2026-03-15T13:58:23.096-07:00	INIT_START Runtime Version: provided:al2.v143 Runtime Version ARN: arn:aws:lambda:us-west-2::runtime:64c84f04bae0287749677ef420f09c0b2f4a3c7b1a638630bcc38cf04c7b	📄
2026-03-15T13:58:23.096-07:00	START RequestId: 287619f7-053b-4f5f-a731-fc208c92051e Version: \$LATEST	📄
2026-03-15T13:58:23.097-07:00	START RequestId: 287619f7-053b-4f5f-a731-fc208c92051e Version: \$LATEST	📄
2026-03-15T13:58:23.097-07:00	2026/03/15 20:58:23 [LAMBDA] Processing order 229f634f-41a6-41d9-9f8a-48b37834fe3f for customer 1	📄
2026-03-15T13:58:23.097-07:00	2026/03/15 20:58:23 [LAMBDA] Processing order 229f634f-41a6-41d9-9f8a-48b37834fe3f for customer 1	📄
2026-03-15T13:58:26.099-07:00	2026/03/15 20:58:26 [LAMBDA] Completed order 229f634f-41a6-41d9-9f8a-48b37834fe3f	📄
2026-03-15T13:58:26.099-07:00	2026/03/15 20:58:26 [LAMBDA] Completed order 229f634f-41a6-41d9-9f8a-48b37834fe3f	📄
2026-03-15T13:58:26.102-07:00	END RequestId: 287619f7-053b-4f5f-a731-fc208c92051e	📄
2026-03-15T13:58:26.102-07:00	END RequestId: 287619f7-053b-4f5f-a731-fc208c92051e	📄
2026-03-15T13:58:26.102-07:00	REPORT RequestId: 287619f7-053b-4f5f-a731-fc208c92051e Duration: 3005.50 ms Billed Duration: 3005 ms Memory Size: 128 MB Max Memory Used: 20 MB Init Duration: 78.68 ms	📄
2026-03-15T13:58:26.102-07:00	REPORT RequestId: 287619f7-053b-4f5f-a731-fc208c92051e Duration: 3005.50 ms Billed Duration: 3005 ms Memory Size: 128 MB Max Memory Used: 20 MB Init Duration: 78.68 ms	📄

No newer events at this moment. Auto retry paused. [Resume](#)

Figure L2 — Cold start invocation (customer 1): INIT_START visible, Init Duration: 78.68 ms. Total billed duration: 3,085 ms.

Timestamp	Message
	No older events at this moment. Retry
2026-03-15T13:58:23.077-07:00	INIT_START Runtime Version: provided:al2.v143 Runtime Version ARN: arn:aws:lambda:us-west-2::runtime:64c84fb4baae0287749677ef42dbf9c0b2bf4a3c7b1a638638bcc38cf04c7b
2026-03-15T13:58:23.151-07:00	INIT_START Runtime Version: provided:al2.v143 Runtime Version ARN: arn:aws:lambda:us-west-2::runtime:64c84fb4baae0287749677ef42dbf9c0b2bf4a3c7b1a638638bcc38cf04c7b
2026-03-15T13:58:23.151-07:00	START RequestId: ec9681fd-6c1d-4348-bd04-9520ff9e6621 Version: \$LATEST
2026-03-15T13:58:23.151-07:00	START RequestId: ec9681fd-6c1d-4348-bd04-9520ff9e6621 Version: \$LATEST
2026-03-15T13:58:23.152-07:00	2026/03/15 20:58:23 [LAMBDA] Processing order 55d3b667-e957-4e41-9ab3-452be254e4fa for customer 5
2026-03-15T13:58:23.152-07:00	2026/03/15 20:58:23 [LAMBDA] Processing order 55d3b667-e957-4e41-9ab3-452be254e4fa for customer 5
2026-03-15T13:58:26.154-07:00	2026/03/15 20:58:26 [LAMBDA] Completed order 55d3b667-e957-4e41-9ab3-452be254e4fa
2026-03-15T13:58:26.154-07:00	2026/03/15 20:58:26 [LAMBDA] Completed order 55d3b667-e957-4e41-9ab3-452be254e4fa
2026-03-15T13:58:26.156-07:00	END RequestId: ec9681fd-6c1d-4348-bd04-9520ff9e6621
2026-03-15T13:58:26.156-07:00	END RequestId: ec9681fd-6c1d-4348-bd04-9520ff9e6621
2026-03-15T13:58:26.156-07:00	REPORT RequestId: ec9681fd-6c1d-4348-bd04-9520ff9e6621 Duration: 3004.55 ms Billed Duration: 3078 ms Memory Size: 128 MB Max Memory Used: 20 MB Init Duration: 73.25 ms
2026-03-15T13:58:26.156-07:00	REPORT RequestId: ec9681fd-6c1d-4348-bd04-9520ff9e6621 Duration: 3004.55 ms Billed Duration: 3078 ms Memory Size: 128 MB Max Memory Used: 20 MB Init Duration: 73.25 ms
	No newer events at this moment. Auto retry paused. Resume

Figure L3 — Cold start invocation (customer 5): Init Duration: 73.25 ms. Processing completed in 3,004 ms.

Timestamp	Message
	No older events at this moment. Retry
2026-03-15T15:03:24.277-07:00	INIT_START Runtime Version: provided:al2.v143 Runtime Version ARN: arn:aws:lambda:us-west-2::runtime:64c84fb4baae0287749677ef42dbf9c0b2bf4a3c7b1a638638bcc38cf04c7b
2026-03-15T15:03:24.370-07:00	INIT_START Runtime Version: provided:al2.v143 Runtime Version ARN: arn:aws:lambda:us-west-2::runtime:64c84fb4baae0287749677ef42dbf9c0b2bf4a3c7b1a638638bcc38cf04c7b
2026-03-15T15:03:24.370-07:00	START RequestId: f1c773ec-6630-408f-b11d-87c2f4564097 Version: \$LATEST
2026-03-15T15:03:24.370-07:00	START RequestId: f1c773ec-6630-408f-b11d-87c2f4564097 Version: \$LATEST
2026-03-15T15:03:24.371-07:00	2026/03/15 22:03:24 [LAMBDA] Processing order f120abac-5c90-4f44-88cb-3c552ed9b3dd for customer 100
2026-03-15T15:03:24.371-07:00	2026/03/15 22:03:24 [LAMBDA] Processing order f120abac-5c90-4f44-88cb-3c552ed9b3dd for customer 100
2026-03-15T15:03:27.373-07:00	2026/03/15 22:03:27 [LAMBDA] Completed order f120abac-5c90-4f44-88cb-3c552ed9b3dd
2026-03-15T15:03:27.373-07:00	2026/03/15 22:03:27 [LAMBDA] Completed order f120abac-5c90-4f44-88cb-3c552ed9b3dd
2026-03-15T15:03:27.375-07:00	END RequestId: f1c773ec-6630-408f-b11d-87c2f4564097
2026-03-15T15:03:27.375-07:00	END RequestId: f1c773ec-6630-408f-b11d-87c2f4564097
2026-03-15T15:03:27.375-07:00	REPORT RequestId: f1c773ec-6630-408f-b11d-87c2f4564097 Duration: 3004.51 ms Billed Duration: 3097 ms Memory Size: 128 MB Max Memory Used: 20 MB Init Duration: 91.79 ms
2026-03-15T15:03:27.375-07:00	REPORT RequestId: f1c773ec-6630-408f-b11d-87c2f4564097 Duration: 3004.51 ms Billed Duration: 3097 ms Memory Size: 128 MB Max Memory Used: 20 MB Init Duration: 91.79 ms
	No newer events at this moment. Auto retry paused. Resume

Figure L4 — Subsequent cold start (customer 100): Init Duration: 91.79 ms. Lambda instance had been recycled between test runs (~15 min idle).

Cold Start Measurements

Invocation	Init Duration	Process Duration	Billed Duration	Memory Used
Customer 1 (cold)	78.68 ms	3,005.50 ms	3,085 ms	20 MB
Customer 5 (cold)	73.25 ms	3,004.55 ms	3,078 ms	20 MB
Customer 100 (cold)	91.79 ms	3,004.51 ms	3,097 ms	20 MB
Warm (theoretical)	0 ms	~3,000 ms	~3,000 ms	20 MB

Key observation: because SNS triggered 5 concurrent Lambda invocations simultaneously, AWS spun up 5 independent container instances — each experiencing its own cold start. Cold start overhead for this Go binary is consistently 73–92 ms, which is negligible compared to the 3,000 ms processing time (~2.5% overhead). For latency-sensitive workloads with sub-100ms SLAs, cold starts would be a more significant concern.

Additionally, Lambda instances were recycled between test sessions (~15 minute idle), causing every new batch to cold-start again. This is expected behavior for low-traffic workloads.

Cost Analysis: ECS vs Lambda

The ECS architecture runs an always-on Fargate task behind an ALB, incurring a fixed monthly cost regardless of order volume. Lambda charges only per invocation and compute duration. Below is a detailed breakdown of when each approach is more economical.

ECS Fargate — Fixed Monthly Cost

Resource	Spec	Unit Cost	Monthly Cost
ECS Task (order service)	0.25 vCPU / 0.5 GB	\$0.04048/vCPU-hr	~\$7.40
ALB	always-on	\$0.008/LCU-hr	~\$5.76
NAT Gateway	always-on	\$0.045/hr	~\$3.84
Total (fixed, regardless of volume)	—	—	~\$17/month

Lambda — Per-Invocation Cost

Each order: 128 MB x 3 seconds = 0.375 GB-seconds of compute.

Cost Component	Free Tier	Rate After Free	Per Order
Requests	1,000,000 req/month	\$0.20 per 1M req	\$0.0000002
Compute (0.375 GB-s)	400,000 GB-s/month	\$0.0000166667/GB-s	\$0.00000625
Total per order (above free tier)	—	—	\$0.00000645

Free tier covers: 400,000 GB-s / 0.375 = 1,066,667 orders/month completely free.

Lambda vs ECS Cost at Different Order Volumes

Orders/Month	Lambda Cost	ECS Cost	Verdict
100,000	\$0.00	\$17.00	Lambda saves \$17.00
500,000	\$0.00	\$17.00	Lambda saves \$17.00
1,066,667 (free tier ends)	\$0.00	\$17.00	Lambda saves \$17.00
2,000,000	\$6.02	\$17.00	Lambda saves \$10.98
3,000,000	\$12.47	\$17.00	Lambda saves \$4.53
3,702,326 (break-even)	\$17.00	\$17.00	Equal cost
5,000,000	\$25.37	\$17.00	ECS saves \$8.37
10,000,000	\$57.62	\$17.00	ECS saves \$40.62

Break-even formula: $\$17 = \$0.00000645 \times (N - 1,066,667) \Rightarrow N = 3,702,326$ orders/month (~123,000 orders/day). Below this threshold Lambda is cheaper; above it ECS wins on cost.

Head-to-Head Comparison

Dimension	ECS + SQS Worker	Lambda (SNS direct)
Monthly cost (startup)	~\$17/month (always-on)	\$0 up to ~1M orders
Scaling	Manual (change NUM_WORKERS)	Automatic per invocation
Cold start	None (always running)	73–92 ms per new instance
Message durability	SQS guarantees retry + DLQ	No queue buffer (SNS direct)
Ops overhead	ECS service, task def, scaling	Zero infra to manage
Max concurrency	Limited by NUM_WORKERS	1000 concurrent (default)
Best for	High sustained throughput	Spiky / low-volume workloads

Recommendation

For an early-stage startup like this one, Lambda is the clear winner. With ~1M orders free per month and zero infrastructure to manage, it eliminates the \$17/month fixed cost entirely during the pre-revenue phase. Auto-scaling handles flash sales without any worker tuning.

The 73–92 ms cold start is negligible for order processing — customers already wait seconds for payment confirmation. The absence of SQS between SNS and Lambda does reduce durability (a Lambda failure loses the message), but this can be mitigated cheaply by adding a dead-letter queue or re-introducing SQS as the Lambda trigger when reliability becomes critical.

ECS + SQS becomes the better choice at sustained high throughput (>1M orders/month) or when strict processing guarantees, retry control, and visibility timeout semantics are required. At that scale, the fixed cost is justified by the operational control SQS provides.

Use Lambda when...	Use ECS + SQS when...
< 1M orders/month (free tier)	> 1M orders/month (cost justifies ECS)
Spiky, unpredictable traffic	Sustained high-throughput processing
Startup / pre-revenue stage	Strict retry / DLQ / ordering guarantees needed
Minimal ops burden preferred	Fine-grained worker concurrency control needed